

Reporte: Actividad 4

“Aplicación de gestión de notas”

Alumnos:

- Juan Carlos Malanco Rodriguez
- Isabel Kyra Romero Borunda
- Jocelyn Flores Gutiérrez

Maestro: Adalberto Emmanuel Rojas Perea

Materia: Desarrollo de Aplicaciones Móviles

Fecha: 12/09/2026

● Índice

● Índice.....	2
● Requerimientos	3
● Diseño	4
● Desarrollo	7
● Uso de IA	11

Desarrollar una aplicación móvil que combine animaciones fluidas, gestos intuitivos y almacenamiento local con Room, priorizando una experiencia de usuario cohesiva y eficiente.

Los equipos crearán una aplicación de gestión de notas que incluya:

Animaciones: transiciones al agregar/eliminar notas, feedback visual al completar tareas.

Gestos: deslizar para eliminar y zoom con pellizco en imágenes adjuntas.

Base de datos local: persistencia de notas usando Room, con operaciones CRUD.

Interfaz adaptable: diseño responsivo para pantallas pequeñas y grandes.

Repositorio Github:

<https://github.com/JoFloresGutierrez06/Actividad-clase-3---DAM.git>

Video:

https://drive.google.com/file/d/1LBuT5r3l4yKak9N7zM3xbFQ8pv_R18fE/view?usp=sharing

● Requerimientos

- *Técnicos:*

- Room + SQLite para almacenar notas (título, contenido, fecha).
- ViewModel + Corrutinas para manejo de datos.
- Animaciones con AnimatedVisibility, animate*AsState o Transition.
- Gestos con Modifier.swipeable o Modifier.pointerInput.

-

- *Diseño:*

- Prototipo en Figma/Adobe XD con especificaciones de animaciones y gestos.
- Paleta de colores accesible (contraste $\geq 4.5:1$).

-

- *Colaboración:*

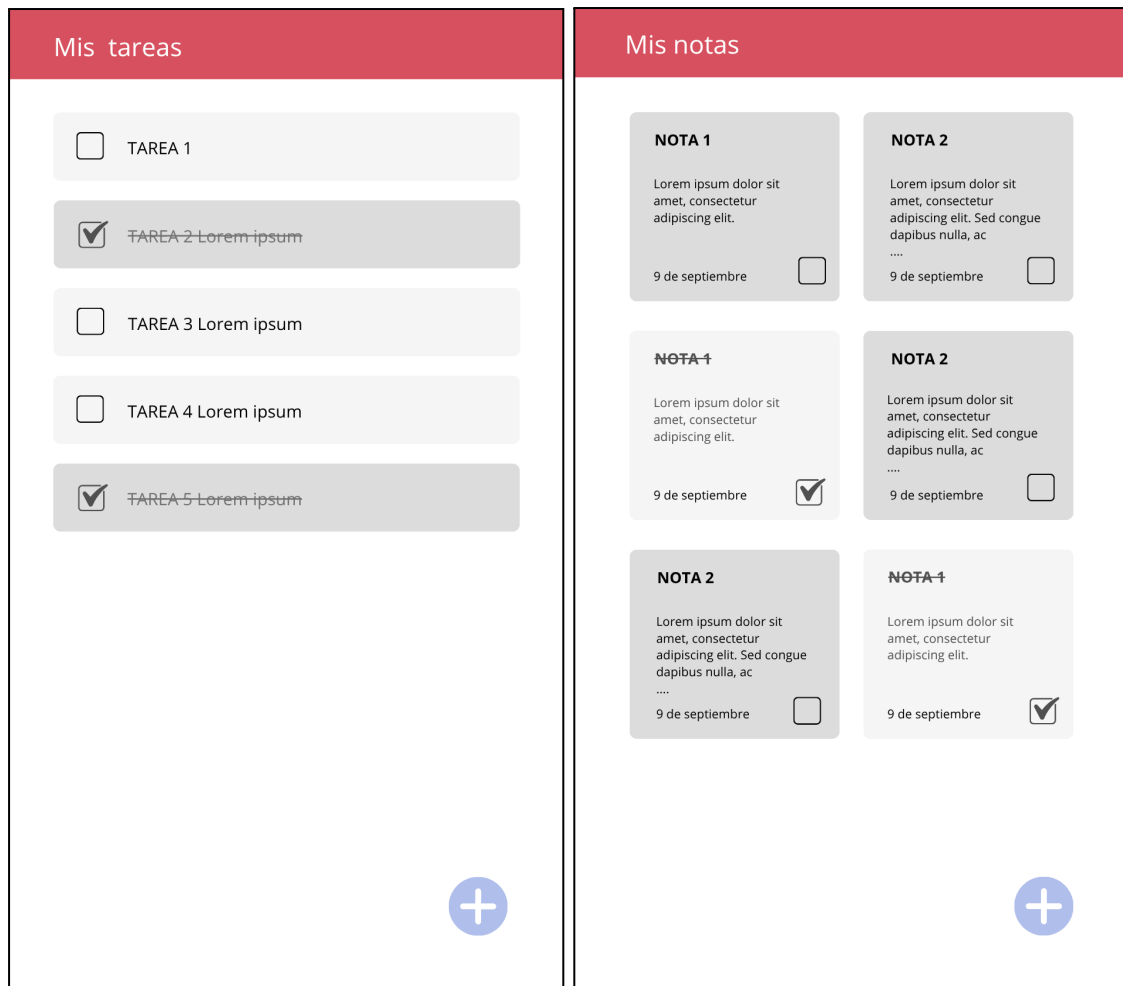
- Uso de Git para integrar cambios entre diseñadores y desarrolladores.
- Documentación de decisiones de diseño y flujos de datos.

● Diseño

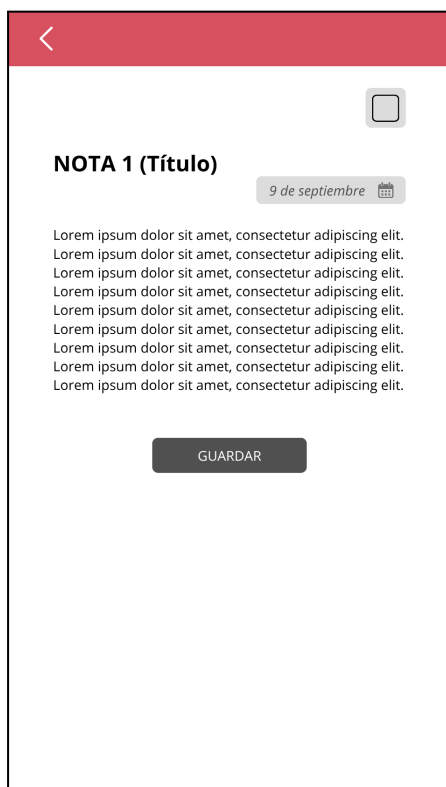
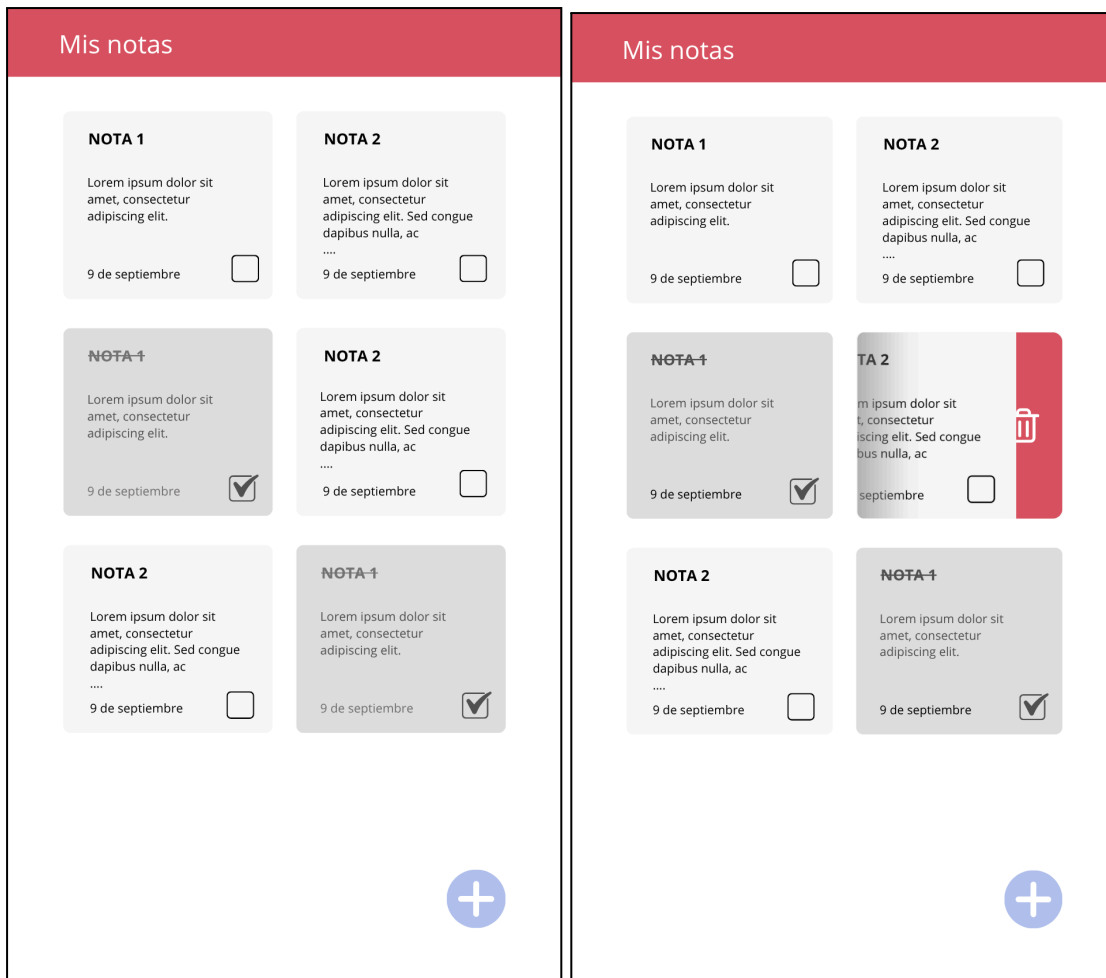
Para esta aplicación se identificó que gran parte de los elementos de una aplicación de gestión de notas son muy similares a los de una lista de tareas, por lo que se buscó modificar la interfaz principal y los componentes más característicos, siguiendo un proceso de diseño con varias propuestas y entrevistas a usuarios para mejorar la experiencia de usuario y priorizar que fuera cómodo e intuitivo.

Nota: No se utilizó Figma, sino Canva para facilitar las distintas versiones de los diseños y dejar un registro de los cambios..

Se partió de la idea de la aplicación de tareas para luego desarrollar una vista de notas.



A partir de aquí se aprobó el cambio, pero según las encuestas de experiencia de usuario se consideró mejor intercambiar el color de las notas completadas y las normales, logrando el siguiente resultado. Luego se decidió conservar la función de eliminarlas simplemente con la acción de desplazarlas, manteniendo por supuesto la animación de tachar las tareas cuando se completen.



Ahora seguía la vista de detalle cuando se le picaba a una nota, debía mostrar y poderse editar el título, la fecha, poder completar o descompletar una tarea, cambiar el contenido de la nota y guardar los cambios (nos encantaría que se guardaran solos con simplemente salirse de la vista, pero por el momento será con un botón).

Se pasaron por varias propuestas de diseño y distribución de los elementos, ganando la que se muestra al final.

Se consideró ahorrar espacio y el movimiento de los elementos cuando el título o el contenido de la nota sean más grandes, por lo que si inicialmente el botón de guardar estaba hasta el final, se movería cuando la nota creciera, entonces se decidió mejor mantenerlo arriba antes del texto.

Se eliminó la opción de eliminar de ahí porque ya está en la vista principal con la acción de deslizar.



NOTA 1. Gestión de proyectos

9 de septiembre

[illegible]

GUARDAR



NOTA 1. Gestión de proyectos

9 de septiembre



Lorem ipsum dolor sit amet, consectetur adipiscing
 elit. Lorem ipsum dolor sit amet, consectetur
 adipiscing elit. Lorem ipsum dolor sit amet,
 consectetur adipiscing elit. Lorem ipsum dolor sit
 amet, consectetur adipiscing elit. Lorem ipsum
 dolor sit amet, consectetur adipiscing elit.

Lorem ipsum dolor sit amet, consectetur adipiscing
 elit. Lorem ipsum dolor sit amet, consectetur
 adipiscing elit. Lorem ipsum dolor sit amet,
 consectetur adipiscing elit. Lorem ipsum dolor sit
 amet, consectetur adipiscing elit.

GUARDAR



GUARDAR

NOTA 1. Gestión de proyectos

9 de septiembre 



Lorem ipsum dolor sit amet, consectetur adipiscing
 elit. Lorem ipsum dolor sit amet, consectetur
 adipiscing elit. Lorem ipsum dolor sit amet,
 consectetur adipiscing elit. Lorem ipsum dolor sit
 amet, consectetur adipiscing elit. Lorem ipsum
 dolor sit amet, consectetur adipiscing elit.

Lorem ipsum dolor sit amet, consectetur adipiscing
 elit. Lorem ipsum dolor sit amet, consectetur
 adipiscing elit. Lorem ipsum dolor sit amet,
 consectetur adipiscing elit. Lorem ipsum dolor sit
 amet, consectetur adipiscing elit.



NOTA 1. Gestión de proyectos

9 de septiembre



Lorem ipsum dolor sit amet, consectetur adipiscing
 elit. Lorem ipsum dolor sit amet, consectetur
 adipiscing elit. Lorem ipsum dolor sit amet,
 consectetur adipiscing elit. Lorem ipsum dolor sit
 amet, consectetur adipiscing elit. Lorem ipsum
 dolor sit amet, consectetur adipiscing elit.

Lorem ipsum dolor sit amet, consectetur adipiscing
 elit. Lorem ipsum dolor sit amet, consectetur
 adipiscing elit. Lorem ipsum dolor sit amet,
 consectetur adipiscing elit. Lorem ipsum dolor sit
 amet, consectetur adipiscing elit.

GUARDAR

● Desarrollo

Explicación de arquitectura

Se implementaron varias cosas al proyecto de acuerdo a los puntos de la actividad, como Room + SQLite para almacenar las notas, ViewModel + Corrutinas para el manejo de datos, animaciones, y varios gestos.

● Manejo de Datos

- Task(Entidad):

En este archivo se crea el objeto de task, donde se usa `@Entity` para decirle a Room que convierta el objeto en una tabla llamada "tasks".

Cada campo de Task vendría siendo una columna que guarda los datos correspondientes, cada tarea necesita un id único para poder identificar cada uno, de eso se encarga `@PrimaryKey`.

```
@Entity(tableName = "tasks")
data class Task(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val title: String,
    val description: String,
    val isCompleted: Boolean = false,
    val dueDate: Long
)
```

- TaskDao(DAO):

El dao vendría siendo como un traductor entre kotlin y la base de datos SQL.

```
@Dao
interface TaskDao {

    1 Usage 1 Implementation
    @Query(value = "SELECT * FROM tasks ORDER BY dueDate ASC")
    fun getAllTasks(): Flow<List<Task>>

    1 Usage 1 Implementation
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertTask(task: Task)

    1 Usage 1 Implementation
    @Update
    suspend fun updateTask(task: Task)

    1 Usage 1 Implementation
    @Delete
    suspend fun deleteTask(task: Task)
}
```

En **Flow<List>Task>>**, Flow permite que la base de datos sea reactiva, si algo cambia en la tabla, Room avisa automáticamente a la interfaz para que esta haga el cambio.

Las funciones de insertar, borrar o editar funcionan con corrutinas(**suspend**), lo que hace que la base de datos trabaje en un hilo secundario.

Por último, **OnConflictStrategy.Replace**, hace que si intentas insertar una tarea con un ID existente, este lo reemplaza por el nuevo.

- **AppDatabase(Base de Datos):**

Este vendría siendo el punto central de acceso y configuración, **companion object** se asegura que solo exista una instancia de la base de datos, **synchronized(this)** evita que dos partes de la app intenten crear la base al mismo tiempo.

```
@Database(entities = [Task::class], version = 1, exportSchema = false)
abstract class AppDatabase : RoomDatabase() {

    1 Usage 1 Implementation
    abstract fun taskDao(): TaskDao

    4 Usages
    companion object {
        2 Usages
        @Volatile
        private var INSTANCE: AppDatabase? = null

        1 Usage
        fun getDatabase(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(lock = this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    class = AppDatabase::class.java,
                    name = "tasks_database"
                ).build()
                INSTANCE = instance
                instance
            }
        }
    }
}
```

- **ViewModel y Corrutinas**

Algunas de las cosas aquí se explicaron en el punto de arriba, el archivo de TaskViewModel.kt actúa como un puente entre el UI y el repositorio, el **StateFlow<List<Task>>** permite que cada vez que haya un cambio en los datos estos se presenten de manera rápida en la UI.

```
5 Usages
private val repository: TaskRepository

2 Usages
val tasks: StateFlow<List<Task>>
```

Luego todas las acciones de escritura y borrado de los datos pasa por **viewModelScope.launch** (corrutinas) que permite pasar el trabajo a un segundo hilo.

```
fun addTask(title: String, description: String, dueDate: Long) {
    viewModelScope.launch {
        repository.insert(
            Task(title = title, description = description, dueDate = dueDate)
        )
    }
}

2 Usages
fun updateTask(task: Task) {
    viewModelScope.launch {
        repository.update(task)
    }
}
```

- **Animaciones**

Se usaron varios tipos de animaciones en la aplicación, como cambios de color en los cuadros al marcarlos como completados, cuando la información de la tarjeta cambia en la vista de la pantalla principal, y por ultimo cuando se elimina una tarea.

Para las animaciones de color color se usó un **animateColorAsState**, y se encuentran a la hora de marcar la nota como completada de un color claro a uno mas oscuro, y cuando se desliza a la izquierda una nota para eliminarla, donde se puede ver el cambio al color rojo.

```
val containerColor by animateColorAsState(
    targetValue = if (task.isCompleted) Color(0xFFE8E8E8) else Color.White,
    animationSpec = tween(durationMillis = 300),
    label = "taskContainerColor"
)
```

```
val color by animateColorAsState(
    targetValue = when (dismissState.dismissDirection) {
        SwipeToDismissBoxValue.EndToStart -> Color.Red
        else -> Color.Transparent
    },
    animationSpec = tween( durationMillis = 200),
    label = "swipeBackgroundColor"
)
```

Luego se usa un **animateContentSize** para que a la hora de que el título de la tarjeta en la vista principal cambia a uno más largo, el cambio de tamaño de la tarjeta no sea tan brusco.

```
Card(
    modifier = Modifier
        .fillMaxWidth()
        .clickable { onTaskClick() }
        .animateContentSize(),
```

También se usa **animateItem** para que al eliminar una nota esta animación sea fluida.

```
SwipeToDismissBox(
    modifier = Modifier
        .animateItem(
            fadeInSpec = tween( durationMillis = 300),
            placementSpec = tween( durationMillis = 300),
            fadeOutSpec = tween( durationMillis = 300)
        ),
```

- **Gestos y Multitouch**

Aquí se agregaron dos tipos de gestos, el de eliminar deslizando a la izquierda y el de hacer zoom a la imagen de perfil.

Para el gesto de eliminación se usó **SwipeToDismissBox**, el cual es un componente que se especializa en escuchar el movimiento horizontal del dedo.

Obviamente se calcula primero que tanto se ha deslizado el contenedor para poder borrarlo, evitando que se borre accidentalmente con un pequeño movimiento.

```
SwipeToDismissBox(  
  modifier = Modifier  
    .animateItem(  
      fadeInSpec = tween( durationMillis = 300),  
      placementSpec = tween( durationMillis = 300),  
      fadeOutSpec = tween( durationMillis = 300)  
    ),  
)
```

Para poder hacer pinch a la imagen se creó una función llamada **ProfileImageDetailDialog**, la cual permite el movimiento y el zoom de esta. Se usó **zoom** para poder acercarla o alejarla y también se empleó **pan** para poder desplazarla hacia los lados. Aparte para lograr esos dos tipos de gestos se usó **pointerInput** el cual es el que detecta lo que se está haciendo en la pantalla.

```
.pointerInput( key1 = Unit) {  
  detectTransformGestures { _, pan, zoom, _ ->  
    scale = (scale * zoom).coerceIn(1f, 5f)  
    offset += pan  
  }  
}
```

Algo importante a mencionar es que para poder transmitir los gestos a movimiento en la imagen se ingresó **graphicsLayer**, que básicamente es lo que multiplica el valor del zoom por el tamaño de la imagen. Para agregarle limite a esto también se usó **corceln**, donde se limito lo máximo a 5x y mínimo 1x.

```
.graphicsLayer(  
  scaleX = scale,  
  scaleY = scale,  
  translationX = offset.x,  
  translationY = offset.y  
)
```

• Uso de IA

Esta vez se le preguntó igual acerca de los errores que iban saliendo, sobre cómo era mejor estructurar algunos archivos, como implementar varias funciones, como cambiar algunos detalles de botones, fields, etc.

Como puedo mostrar la fecha en el boton?

Thought

Read [TaskDetailScreen.kt](#)

Poque sale error ?

Thought

lyzed TaskListScreen.kt

Me refiero en mainactivity, o es mejor haacerlo en un archivo diferente?

Thought

Es mucho mejor **hacerlo en un archivo diferente**, tal como lo tienes ahora en [TaskListScreen.kt](#).

En el desarrollo profesional de Android, se sigue el principio de **Responsabilidad Única**:

1. [MainActivity.kt](#): Debe actuar solo como el "director de orquesta". Se encarga de:
 - Configurar la navegación (`NavHost`).
 - Conectar el `Navigation` con las pantallas.

El error ocurría principalmente por dos razones después de intentar añadir la funcionalidad de "deslizar para eliminar" (Swipe to Dismiss):

1. **Faltaba el parámetro `onDeleteTask`**: En [TaskListScreen](#) se estaba intentando llamar a `onDeleteTask(task)`, pero esa función no estaba definida en los parámetros de la pantalla ni se estaba pasando desde la [MainActivity](#).
2. **Faltaban las dependencias e importaciones de Icons**: Se estaba usando `Icons.Default.Delete`, pero no estaba importado y la librería de iconos extendidos no estaba en el proyecto.

Lo pongo dentro aunque ya este dentro de otro componente?

Thought

Sí, ¡exactamente! En Compose, la **anidación** (poner un componente dentro de otro) es la forma estándar de trabajar.

Dependiendo de qué quieras que se anime exactamente, tienes dos opciones principales en tu archivo [TaskListScreen.kt](#):

Opción A: Envolver la tarjeta completa (Lo más común)

Si quieres que toda la fila (incluyendo la capacidad de

Para mostrar la fecha en el botón de forma legible, necesitas usar un formateador (como el `dateFormatter` que ya tienes definido) y pasarle el valor `dueDate`.

He corregido tu archivo [TaskDetailScreen.kt](#) porque tenías algunos errores de diseño (un `Row` que ocupaba toda la pantalla y una errata que decía `test` en lugar de `text`).

Aquí tienes el código clave para mostrar la fecha en un botón:

```
Kotlin
// Dentro de tu Column en TaskDetailScreen.kt
Button(
    onClick = { showDatePicker = true }, // AL
    modifier = Modifier.fillMaxWidth()
) {
    // Usamos el formateador para convertir el
    Text(text = "Fecha: ${dateFormatter.format(
    }
    }
```

Como le puedo cabiar el color y la forma a boton?, que se usa para hacerlo cuadrado con lados suaves y de diferente color en el boto y el texto

Thought

Read [TaskDetailScreen.kt](#)

Thought

- **Algunas soluciones dadas**

1.-

```
// Dentro de tu Column en TaskDetailScreen.kt
Button(
    onClick = { showDatePicker = true }, // Al pulsar, abre el calendario
    modifier = Modifier.fillMaxWidth()
){
    // Usamos el formateador para convertir el número (Long) en texto "dd/MM/yyyy"
    Text(text = "Fecha: ${dateFormatter.format(dueDate)}")
}
```

2.-

```
Button(
    onClick = { /* acción */ },
    modifier = Modifier.fillMaxWidth(),
    // 1. FORMA: Cuadrado con esquinas redondeadas suavemente
    shape = RoundedCornerShape(8.dp),
    // 2. COLORES: Fondo morado y texto blanco
    colors = ButtonDefaults.buttonColors(
        containerColor = Color(0xFF6200EE), // Fondo
        contentColor = Color.White // Texto
    )
){
    Text("Guardar cambios")
}
```

3.-

```
OutlinedTextField(
    value = description,
    onValueChange = { description = it },
    modifier = Modifier.fillMaxWidth(),
    // 1. FORMA: Esquinas redondeadas suaves (igual que tus botones)
    shape = RoundedCornerShape(8.dp),
    // 2. COLORES: Fondo gris muy claro y bordes personalizados
    colors = OutlinedTextFieldDefaults.colors(
        focusedContainerColor = Color(0xFFFFF5F5), // Fondo al escribir
        unfocusedContainerColor = Color(0xFFFFF5F5), // Fondo normal
        focusedBorderColor = Color(0xFF808080), // Borde al escribir (Gris oscuro)
        unfocusedBorderColor = Color(0xFFD2D2D2) // Borde normal (Gris claro)
    )
)
```